

Тестовое задание Bash-скрипт

Цель:

Разработать сервис на Go, который умеет удаленно подключаться к Linux-хостам по SSH, размещать на них скрипты по шаблонам, устанавливать агент для отслеживания действий с этими скриптами и принимать события от агента.

Требования:

Код выложить на GitHub!

Необходимо реализовать приложение на Go, которое предоставляет HTTP API на двух различных портах.

1. API №1 (порт 8080)

Предназначен для создания скриптов на удаленной машине.

Пример запроса:

```
POST /create
{
  "host": "192.168.1.10",
  "user": "root",
  "password": "password",
  "template": "template1"
}
```

где:

- host — IP-адрес Linux-машины;
- user — пользователь для SSH;
- password — пароль SSH;
- template — имя шаблона (template1 или template2).

Что должен сделать сервис в результате:

- Подключиться к указанной машине по SSH.
- Создать bash-скрипт на удаленной машине согласно выбранному шаблону.
- Развернуть (или запустить) агент мониторинга.
- Вернуть успешный ответ клиенту.

Шаблоны:

Должно быть реализовано минимум два различных шаблона bash-скрипта.

Например:

1. template1

```
#!/bin/bash
```

```
echo "Template 1"
```

```
date
```

```
hostname
```

2. template2

```
#!/bin/bash
```

```
echo "Template 2"
```

```
whoami
```

```
pwd
```

Шаблоны могут храниться любым удобным способом (файлы, embed, строки в коде и т.д.).

Агент:

После установки агент должен отслеживать действия со сгенерированными скриптами.

Необходимо определить следующие события:

- открытие файла;
- выполнение файла.

При возникновении события агент должен отправить HTTP Callback на API сервиса.

Формат callback может быть произвольным, но должен содержать:

- имя пользователя;
- имя скрипта;

- время события;
 - тип события (open или execute).
3. API №2 (порт 8081)

Предназначен для приема callback от агента.

Пример тела запроса:

```
POST /callback

{
  "user": "root",
  "script": "/tmp/test.sh",
  "action": "execute",
  "time": "2026-07-08T12:30:15Z"
}
```

После получения callback сервис должен:

- вывести информацию в консоль;
- вернуть HTTP 200.

Например:

- [root]
- Action: execute
- Script: /tmp/test.sh
- Time: 2026-07-08T12:30:15Z

3. Дополнительное задание

Добавить поддержку базы данных.

Необходимо сохранять:

- удаленную машину;
- созданный скрипт;
- пользователя;
- время создания;
- все события, полученные от агента.

Тип БД - PostgreSQL любой версии

Будет плюсом:

- Docker / Docker Compose;
- конфигурация через ENV или YAML;
- логирование;
- graceful shutdown;
- unit-тесты;
- Makefile;
- использование ORM или миграций;
- использование embed для хранения шаблонов.